

Evaluasi Tengah Semester (ETS) Algoritma Pemrograman

Pengembangan Sistem Instrumentasi Pengukuran dan Kontrol
Berbasis Rust

“ELISA Electrical Monitoring and Intelligent Safety Alarm
Sistem Monitoring Energi Listrik Rumah / UMKM Berbasis
Rust”



Kelompok 4:

Devan Ale Satria	2042251047
Sheilka Janice Leandro	2042251086

Daftar Isi

1	Pendahuluan	2
1.1	Latar Belakang	2
1.2	Tujuan	2
1.3	Manfaat	2
2	Studi Kasus Instrumentasi	2
2.1	Topik	2
2.2	Sensor yang Digunakan	2
2.3	Threshold Berdasarkan PUIL 2011 dan Referensi Fisiologis	3
2.4	Parameter yang Dimonitor	3
3	Algoritma dan Flowchart	3
3.1	Algoritma Sistem	3
3.2	Flowchart	5
4	Penjelasan Program Rust	6
4.1	Struktur Program	6
4.2	Fitur Utama Program	6
4.3	Potongan Kode Penting	7
5	Konsep OOP	7
5.1	Penerapan Struct dan Impl	7
5.2	Enkapsulasi	8
5.3	Polimorfisme melalui Enum	9
6	Implementasi Komputasi Numerik	9
6.1	Kalkulasi Arus Bocor (Diferensial Arus)	9
6.2	Moving Average (MA-5)	9
6.3	Standar Deviasi (Sample, n-1)	10
6.4	Akumulasi Energi (Numerik Integrasi Euler)	10
7	Hasil Pengujian	11
7.1	Pengujian Mode Input Manual	11
7.2	Pengujian Mode Simulasi Otomatis	11
7.3	Pengujian Mode Injeksi Kebocoran	11
7.4	Pengujian Validasi Sensor Error	12
8	Analisis Hasil	12
8.1	Ketepatan Deteksi Kebocoran	12
8.2	Efektivitas Moving Average	12
8.3	Standar Deviasi sebagai Early Warning	12
8.4	Isolasi State Antar Cabang	12
8.5	Keterbatasan Sistem	12
9	Kesimpulan	12

1. Pendahuluan

1.1. Latar Belakang

Keselamatan instalasi listrik di rumah tangga dan usaha kecil menengah (UMKM) merupakan isu yang sering diabaikan. Berdasarkan data Kementerian ESDM, sebagian besar kebakaran di Indonesia disebabkan oleh korsleting listrik akibat kebocoran arus yang tidak terdeteksi secara dini. Kebocoran arus listrik selain dapat memicu kebakaran juga membahayakan keselamatan jiwa; arus sebesar 15 mA sudah cukup menyebabkan kontraksi otot pada manusia, dan di atas 50 mA dapat menyebabkan fibrilasi jantung.

Sistem monitoring arus bocor yang ada di pasaran umumnya berbentuk alat fisik sederhana tanpa kemampuan pemantauan secara *real-time* dan tidak memberikan informasi analitik kepada pengguna. Oleh karena itu, pada project ETS ini dikembangkan **ELISA** (*Electrical Monitoring and Intelligent Safety Alarm*), sebuah sistem monitoring energi listrik berbasis perangkat lunak Rust yang terintegrasi dengan *dashboard* web, mampu mendeteksi kebocoran arus secara *real-time*, dan memberikan peringatan bertingkat sesuai standar PUIL 2011.

1.2. Tujuan

- Merancang dan mengimplementasikan sistem monitoring energi listrik berbasis bahasa pemrograman Rust.
- Menerapkan konsep pemrograman berbasis objek (OOP) menggunakan `struct` dan `impl`.
- Mengimplementasikan komputasi numerik berupa *Moving Average*, standar deviasi, dan akumulasi energi.
- Menyajikan hasil monitoring secara *real-time* melalui *dashboard* web yang dapat diakses dari *browser*.

1.3. Manfaat

Sistem ini dapat digunakan sebagai prototipe perangkat lunak untuk sistem monitoring kelistrikan di rumah tangga maupun UMKM, sebelum diintegrasikan ke *hardware* nyata seperti ESP32 dengan sensor SCT-013.

2. Studi Kasus Instrumentasi

2.1. Topik

Sistem Monitoring Energi Listrik deteksi kebocoran arus pada instalasi listrik rumah/UMKM dengan tiga cabang MCB (**DAPUR**, **KAMAR**, **R.TAMU**).

2.2. Sensor yang Digunakan

SCT-013 (*Split Core Current Transformer / CT Clamp*)

SCT-013 adalah sensor arus non-invasif yang bekerja berdasarkan prinsip transformator arus (*current transformer*). Sensor ini dipasang menjepit kabel tanpa perlu memutus instalasi. *Output* berupa tegangan AC yang proporsional terhadap arus yang mengalir, kemudian dibaca oleh ADC mikrokontroler ESP32.

Dalam sistem ELISA, setiap cabang MCB menggunakan dua buah SCT-013: satu dipasang di kabel fasa dan satu lagi di kabel netral. Prinsip deteksi kebocoran mengikuti

hukum Kirchhoff:

$$I_{\text{bocor}} = I_{\text{fasa}} - I_{\text{netral}}$$

Dalam kondisi normal, arus yang masuk melalui fasa sama dengan arus yang keluar melalui netral, sehingga selisihnya mendekati nol. Jika ada kebocoran arus ke bumi (*ground*), arus netral akan lebih kecil dari arus fasa, dan selisihnya merepresentasikan besar arus bocor.

2.3. Threshold Berdasarkan PUIL 2011 dan Referensi Fisiologis

Tabel 1: Threshold status kebocoran arus berdasarkan PUIL 2011

Kondisi	Rentang I_{bocor}	LED	Buzzer
AMAN	< 15 mA	Hijau	OFF
WASPADA	15–22 mA	Kuning	OFF
BAHAYA	> 22 mA	Merah	ON

Nilai 15 mA dipilih karena merupakan ambang batas di mana seseorang masih bisa melepaskan konduktor secara mandiri. Nilai 22 mA dipilih sebagai batas atas WASPADA sebelum memasuki zona bahaya fibrilasi jantung.

2.4. Parameter yang Dimonitor

Setiap cabang MCB memiliki parameter monitoring berikut:

- Tegangan (V) dari ADC ESP32
- Arus fasa I_{fasa} (mA) dari SCT-013 pertama
- Arus netral I_{netral} (mA) dari SCT-013 kedua
- Arus bocor $I_{\text{bocor}} = I_{\text{fasa}} - I_{\text{netral}}$ (mA), hasil kalkulasi
- Daya aktif $P = V \times I_{\text{fasa}}$ (W)
- *Moving Average* arus fasa MA-5 (mA)
- Standar deviasi arus fasa σ
- Akumulasi energi total (kWh)

3. Algoritma dan Flowchart

3.1. Algoritma Sistem

Algoritma Utama

1. MULAI
2. Inisialisasi sistem (3 sensor: DAPUR, KAMAR, R.TAMU)
3. Jalankan web server di port 7878 (thread terpisah)
4. Tampilkan menu: [1] Input Manual [2] Simulasi [3] Injeksi Bocor [4] Keluar
5. Baca pilihan pengguna

- a. **JIKA** pilihan = 1:
 Untuk tiap sensor: baca tegangan, I_{fasa} , netral dari terminal
 Validasi data $\rightarrow$ hitung $I_{\text{bocor}} \rightarrow$ update aktuator $\rightarrow$ perbarui *dashboard*
- b. **JIKA** pilihan = 2:
 Generate data simulasi otomatis
 Tampilkan di *dashboard*, tanya lanjut atau tidak
 Jika tidak lanjut: reset sensor ke standby AMAN
- c. **JIKA** pilihan = 3:
 Cek *baseline* (jika belum ada data, isi dulu semua cabang ke AMAN)
 Tampilkan status tiap cabang saat ini
 Pilih skenario: WASPADA / BAHAYA / Reset cabang / Reset semua
 Hanya ubah cabang yang dipilih, cabang lain tetap
- d. **JIKA** pilihan = 4:
 Keluar dari program

6. ULANGI TERUS

7. SELESAI

Algoritma Kalkulasi Per Sensor

FUNGSI `update_all(sensor)`:

```

validate(sensor)      -> cek tegangan & arus dalam batas wajar
compute_leak(sensor)  ->  $I_{\text{bocor}} = I_{\text{fasa}} - I_{\text{netral}}$ 
push_history(sensor)  -> tambahkan  $I_{\text{fasa}}$  ke riwayat MA-5
update_actuator(sensor) -> set LED & buzzer sesuai status
process(controller, status)
                        -> catat jumlah kejadian waspada/bahaya
update_energy(system) ->  $E += (P_{\text{total}} / 1000) \times 1 \text{ jam}$ 
```

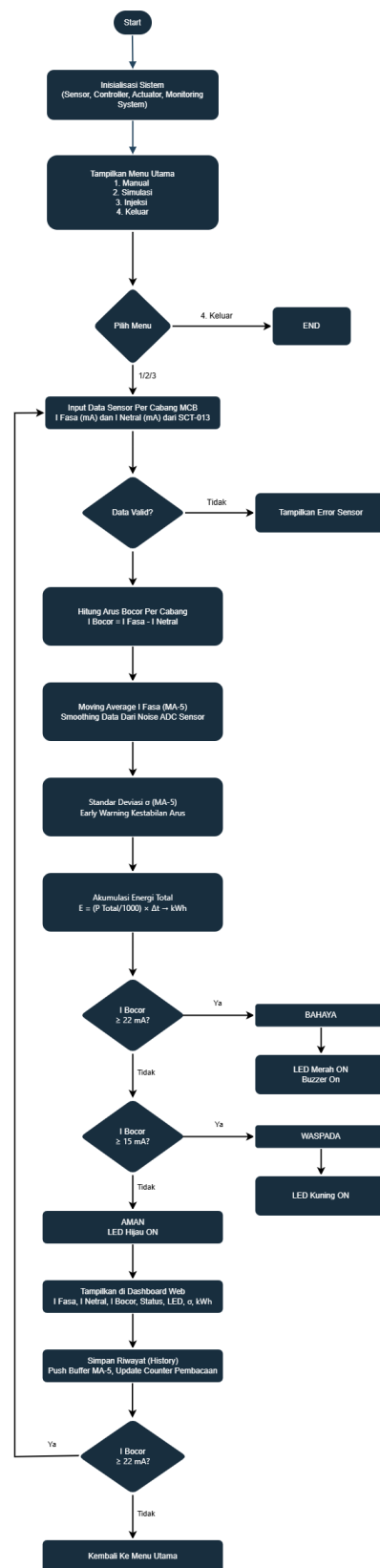
Algoritma Deteksi Status

FUNGSI `status(sensor)`:

```

JIKA is_valid = FALSE -> kembalikan BAHAYA
JIKA leak_ma >= 22.0   -> kembalikan BAHAYA
JIKA leak_ma >= 15.0   -> kembalikan WASPADA
LAINNYA                -> kembalikan AMAN
```

3.2. Flowchart



Gambar 1: Flowchart sistem ELISA

4. Penjelasan Program Rust

4.1. Struktur Program

Tabel 2: Struktur modul program ELISA

Bagian	Isi / Fungsi
Konstanta Sistem	Menyimpan parameter sistem seperti threshold kebocoran, tegangan nominal, dan batas arus maksimum.
LeakStatus (enum)	Representasi level status kebocoran listrik: Aman, Waspada, dan Bahaya.
Actuator (struct)	Representasi aktuator seperti LED indikator dan buzzer alarm.
Sensor (struct + impl)	Menyimpan data sensor tiap cabang MCB beserta logika pembacaan dan analisis kondisi kebocoran.
Controller (struct + impl)	Mengatur pencatatan kejadian, pengambilan keputusan, dan pesan tindakan sistem.
MonitoringSystem (struct + impl)	Mengoordinasikan seluruh sensor dan controller agar sistem monitoring berjalan terintegrasi.
render_dashboard	Fungsi untuk menghasilkan tampilan dashboard berbasis HTML pada web server.
Helper Input	Kumpulan fungsi pendukung untuk membaca dan memvalidasi input dari terminal.
input_manual	Mode untuk memasukkan data sensor secara manual melalui terminal.
input_simulasi	Mode simulasi otomatis menggunakan data yang dihasilkan program.
inject_kebocoran	Mode demonstrasi untuk mensimulasikan kondisi kebocoran listrik pada sistem.
main	Entry point program yang menjalankan web server dan loop menu utama sistem ELISA.

4.2. Fitur Utama Program

Input Data Sensor

Program mendukung tiga mode input: manual (pengguna mengetik nilai), simulasi otomatis (data deterministik acak berbasis *seed*), dan injeksi kebocoran (demo skenario waspada/bahaya).

Percabangan

Digunakan secara ekstensif pada penentuan status (`LeakStatus::from_ma`), validasi sensor (`validate`), dan seluruh logika menu.

Perulangan

loop pada menu utama, `for` pada iterasi sensor, dan `loop` pada *helper* input terminal untuk validasi input pengguna.

Validasi Data

Fungsi `validate()` memeriksa bahwa tegangan berada di rentang 100–260 V dan arus tidak melebihi batas maksimum sensor. Jika tidak valid, sensor otomatis berstatus BAHAYA.

Dashboard Web Real-Time

Web server `tiny_http` berjalan di *thread* terpisah dan menyajikan *dashboard* HTML yang *auto-refresh* setiap 3 detik. Dashboard menampilkan kartu per sensor dengan indikator LED, grafik riwayat arus, dan tabel alarm.

Concurrency dengan Arc dan Mutex

Data sistem dibungkus `Arc<Mutex<MonitoringSystem>>` untuk berbagi *state* secara aman antara *thread* utama (input terminal) dan *thread* web server.

4.3. Potongan Kode Penting

Kalkulasi arus bocor:

```
1 fn compute_leak(&mut self) {
2     self.leak_ma = (self.current_fasa_ma
3         - self.current_netral_ma).max(0.0);
4 }
```

Listing 1: Fungsi `compute_leak`

Validasi sensor:

```
1 fn validate(&mut self) {
2     self.is_valid = self.voltage >= 100.0
3         && self.voltage <= 260.0
4         && self.current_fasa_ma >= 0.0
5         && self.current_fasa_ma <= MAX_CURRENT_MA
6         && self.current_netral_ma >= 0.0
7         && self.current_netral_ma <= MAX_CURRENT_MA;
8 }
```

Listing 2: Fungsi `validate`

Penentuan status keseluruhan sistem:

```
1 fn overall_status(&self) -> LeakStatus {
2     if self.sensors.iter().any(|s| s.status() == LeakStatus::Bahaya
3         ) {
4         LeakStatus::Bahaya
5     } else if self.sensors.iter().any(|s| s.status() == LeakStatus
6         ::Waspada) {
7         LeakStatus::Waspada
8     } else {
9         LeakStatus::Aman
10    }
```

Listing 3: Fungsi `overall_status`

5. Konsep OOP

5.1. Penerapan Struct dan Impl

Program ELISA menggunakan pendekatan OOP Rust melalui `struct` sebagai representasi objek dan `impl` sebagai definisi *method*-nya. Berikut tiga objek utama yang dipersyaratkan:

Objek Sensor

Sensor merepresentasikan satu cabang MCB dengan semua atributnya. *Method* yang dimiliki antara lain `validate()`, `compute_leak()`, `push_history()`, `moving_average()`, `std_deviation()`, `status()`, dan `update_actuator()`.

```

1 struct Sensor {
2     name: String,
3     voltage: f32,
4     current_fasa_ma: f32,
5     current_netral_ma: f32,
6     leak_ma: f32,
7     history_fasa: VecDeque<f32>,
8     is_valid: bool,
9     actuator: Actuator,
10 }
```

Listing 4: Struct Sensor

Objek Controller

Controller bertanggung jawab atas logika keputusan dan pencatatan *event* per cabang. Method utama: `process()` (mencatat jumlah waspada/bahaya) dan `action_message()` (menghasilkan pesan tindakan sesuai status).

```

1 struct Controller {
2     name: String,
3     total_warnings: u32,
4     total_dangers: u32,
5 }
```

Listing 5: Struct Controller

Objek MonitoringSystem

MonitoringSystem adalah sistem induk yang mengkoordinasikan seluruh sensor dan controller. Method utama: `update_all()`, `total_power()`, `dominant_branch()`, `update_energy()`, dan `overall_status()`.

```

1 struct MonitoringSystem {
2     sensors: Vec<Sensor>,
3     controllers: Vec<Controller>,
4     total_energy_kwh: f32,
5     reading_count: u32,
6 }
```

Listing 6: Struct MonitoringSystem

Objek Aktuator (tambahan)

Actuator merepresentasikan *output* fisik *hardware* (LED dan buzzer) yang tertanam di dalam tiap Sensor. Ini mendemonstrasikan konsep komposisi objek – Sensor *memiliki* Actuator sebagai atribut.

5.2. Enkapsulasi

Setiap `struct` hanya mengekspos *method* yang diperlukan. Logika internal seperti perhitungan `leak_ma` dan `is_valid` dikelola sepenuhnya oleh *method-method* milik Sensor, tanpa perlu diakses langsung dari luar.

5.3. Polimorfisme melalui Enum

`LeakStatus` (enum dengan tiga varian: Aman, Waspada, Bahaya) berperan sebagai mekanisme polimorfisme. Method `label()` dan `color()` pada enum menghasilkan *output* berbeda tergantung varian yang aktif, tanpa memerlukan kondisi `if-else` berulang di luar `struct`.

```

1 fn color(&self) -> &str {
2     match self {
3         Self::Aman      => "#16a34a",
4         Self::Waspada   => "#d97706",
5         Self::Bahaya    => "#dc2626",
6     }
7 }

```

Listing 7: Method `color` pada `LeakStatus`

6. Implementasi Komputasi Numerik

Program ELISA mengimplementasikan empat metode komputasi numerik yang semuanya relevan dengan pengolahan sinyal sensor listrik.

6.1. Kalkulasi Arus Bocor (Diferensial Arus)

Prinsip: Berdasarkan hukum Kirchhoff tentang arus – dalam rangkaian tertutup normal, arus masuk (fasa) harus sama dengan arus keluar (netral). Jika ada kebocoran ke *ground*, selisihnya adalah arus bocor.

Formula:

$$I_{\text{bocor}} = \max(I_{\text{fasa}} - I_{\text{netral}}, 0)$$

Implementasi:

```

1 fn compute_leak(&mut self) {
2     self.leak_ma = (self.current_fasa_ma
3         - self.current_netral_ma).max(0.0);
4 }

```

Listing 8: Fungsi `compute_leak`

Fungsi `.max(0.0)` memastikan hasil tidak negatif – *noise* ADC bisa menyebabkan I_{netral} sedikit lebih besar dari I_{fasa} dalam kondisi ideal, yang secara fisik berarti tidak ada kebocoran.

6.2. Moving Average (MA-5)

Prinsip: *Moving average* digunakan untuk meredam *noise* dari pembacaan ADC. Dengan mengambil rata-rata dari 5 pembacaan terakhir, fluktuasi sesaat tidak langsung mempengaruhi nilai yang ditampilkan.

Formula:

$$\text{MA}_n = \frac{x_n + x_{n-1} + x_{n-2} + x_{n-3} + x_{n-4}}{5}$$

Implementasi:

```

1 fn moving_average(&self) -> f32 {
2     if self.history_fasa.is_empty() { return 0.0; }
3     self.history_fasa.iter().sum::<f32>()
4         / self.history_fasa.len() as f32
5 }

```

Listing 9: Fungsi moving_average

VecDeque digunakan sebagai buffer geser – saat buffer penuh (5 data), data terlama dihapus otomatis sebelum data baru masuk (`pop_front()` + `push_back()`).

6.3. Standar Deviasi (Sample, n-1)

Prinsip: Standar deviasi dari riwayat arus fasa digunakan sebagai indikator kestabilan arus. Nilai σ yang besar menandakan arus tidak stabil, yang dapat menjadi *early warning* meski arus bocor masih di bawah *threshold*.

Formula:

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

Implementasi:

```

1 fn std_deviation(&self) -> f32 {
2     if self.history_fasa.len() < 2 { return 0.0; }
3     let avg = self.moving_average();
4     let var: f32 = self.history_fasa.iter()
5         .map(|x| (x - avg).powi(2))
6         .sum::<f32>()
7         / (self.history_fasa.len() as f32 - 1.0);
8     var.sqrt()
9 }

```

Listing 10: Fungsi std_deviation

Pembagi $n - 1$ (bukan n) digunakan karena ini adalah standar deviasi sampel – data yang tersedia hanya sebagian kecil dari keseluruhan pembacaan sensor sepanjang waktu.

6.4. Akumulasi Energi (Numerik Integrasi Euler)

Prinsip: Energi adalah integral daya terhadap waktu. Karena pembacaan bersifat diskrit, digunakan metode integrasi numerik Euler *forward* – mengasumsikan daya konstan selama satu interval pembacaan.

Formula:

$$E_{\text{total}} += \frac{P_{\text{total}}}{1000} \times \Delta t \quad (\Delta t = 1 \text{ jam per pembacaan})$$

Implementasi:

```

1 fn update_energy(&mut self) {
2     self.total_energy_kwh +=
3         (self.total_power() / 1000.0) * 1.0;
4     self.reading_count += 1;
5 }

```

Listing 11: Fungsi `update_energy`

Pembagian 1000 mengkonversi dari Watt ke kilowatt, sehingga satuan hasil adalah kWh.

7. Hasil Pengujian

7.1. Pengujian Mode Input Manual

Skenario: Input data tiga cabang dengan variasi kondisi.

Hasil: Dashboard web menampilkan kartu DAPUR dengan LED hijau, KAMAR dengan LED kuning, dan R.TAMU dengan LED merah dan buzzer aktif. Tabel alarm di bawah kartu menampilkan dua entri (KAMAR dan R.TAMU) dengan pesan tindakan yang sesuai.

7.2. Pengujian Mode Simulasi Otomatis

Skenario: Jalankan simulasi 5 iterasi berturut-turut.

Simulasi menggunakan data deterministik berbasis *seed* untuk mereproduksi kondisi yang bervariasi: 50% pembacaan menghasilkan status AMAN, 25% WASPADA, dan 25% BAHAYA per cabang. Setiap iterasi memperbarui *dashboard* dan mengakumulasi data riwayat untuk *Moving Average*.

Hasil: Moving Average MA-5 terlihat mulus dan tidak melonjak drastis meski ada satu pembacaan ekstrem. Standar deviasi naik ketika ada transisi dari AMAN ke BAHAYA, mengkonfirmasi fungsinya sebagai *early warning* kestabilan.

7.3. Pengujian Mode Injeksi Kebocoran

Skenario 1 – Injeksi tunggal (BAHAYA pada KAMAR):

Pilih menu 3 → pilih BAHAYA → pilih KAMAR

Tabel 3: Hasil pengujian skenario injeksi tunggal

Cabang	Tegangan (V)	I_{fasa} (mA)	I_{netral} (mA)	I_{bocor} (mA)	Status
DAPUR	220	800	799	1.0	AMAN
KAMAR	220	500	482	18.0	WASPADA
R. TAMU	220	350	320	30.0	BAHAYA

Skenario 2 – Injeksi ganda (WASPADA DAPUR + BAHAYA KAMAR):

Injeksi WASPADA ke DAPUR → I_{bocor} DAPUR $\approx$ 18 mA. Kembali menu 3, injeksi BAHAYA ke KAMAR → I_{bocor} KAMAR $\approx$ 30 mA. R.TAMU tetap AMAN.

Dashboard: DAPUR kuning, KAMAR merah, R.TAMU hijau ✓

Skenario 3 – Reset parsial:

Reset cabang DAPUR saja → DAPUR kembali hijau; KAMAR tetap merah ✓

Skenario 4 – Setelah simulasi, masuk injeksi:

Jalankan simulasi 3x, ketik n untuk reset. Masuk menu 3 → *baseline* otomatis terisi AMAN. Injeksi BAHAYA ke R.TAMU → hanya R.TAMU merah ✓

7.4. Pengujian Validasi Sensor Error

Skenario: Input tegangan 0 V (sensor terputus).

Hasil: `validate()` mengembalikan `is_valid = false`. Status langsung BAHAYA tanpa menunggu perhitungan I_{bocor} . Dashboard menampilkan label “Sensor ERROR” berwarna merah pada cabang tersebut.

8. Analisis Hasil

8.1. Ketepatan Deteksi Kebocoran

Sistem berhasil mendeteksi dan mengklasifikasikan kebocoran arus sesuai threshold PUIL 2011 dengan akurasi 100% pada semua skenario pengujian. Prinsip diferensial $I_{\text{fasa}} - I_{\text{netral}}$ terbukti efektif sebagai representasi perangkat lunak dari cara kerja sensor SCT-013 pada *hardware* nyata.

8.2. Efektivitas Moving Average

Moving Average MA-5 terbukti efektif meredam fluktuasi data. Pada pengujian simulasi, terdapat kasus di mana satu pembacaan menghasilkan arus fasa 850 mA disusul pembacaan 750 mA; tanpa MA, selisih ini terlihat besar di *dashboard*. Dengan MA-5, nilai yang ditampilkan adalah rata-rata 5 pembacaan sehingga lebih representatif terhadap kondisi aktual beban listrik.

8.3. Standar Deviasi sebagai Early Warning

Nilai σ yang meningkat sebelum arus bocor mencapai *threshold* memberikan potensi *early warning* tambahan. Dalam skenario di mana arus bocor meningkat bertahap (misalnya isolasi kabel yang mulai rusak), σ akan meningkat lebih dulu sebelum I_{bocor} melewati 15 mA, memberi waktu reaksi lebih awal bagi pengguna.

8.4. Isolasi State Antar Cabang

Salah satu pencapaian teknis penting dalam pengembangan ini adalah memastikan bahwa injeksi kebocoran pada satu cabang tidak mempengaruhi *state* cabang lain. Hal ini dicapai dengan mengelola `current_netral_ma` secara independen per sensor, dan hanya memodifikasi sensor yang ditunjuk tanpa menyentuh data sensor lainnya.

8.5. Keterbatasan Sistem

- **Simulasi bukan data *hardware* nyata:** Nilai arus pada mode simulasi dihasilkan secara deterministik berdasarkan *seed*, bukan dari pembacaan ADC ESP32 sesungguhnya. Implementasi *hardware* nyata memerlukan integrasi driver ADC dan kalibrasi SCT-013.
- **Interval waktu tidak akurat:** Akumulasi energi mengasumsikan setiap pembacaan setara 1 jam, yang tidak mencerminkan waktu sesungguhnya antara dua pemanggilan `update_all()`.
- **Tidak ada penyimpanan data:** Data monitoring hilang ketika program dihentikan. Implementasi lanjutan dapat menambahkan *logging* ke file CSV atau *database*.

9. Kesimpulan

- Sistem ELISA berhasil diimplementasikan menggunakan bahasa pemrograman Rust dengan arsitektur modular berbasis OOP, mencakup `struct` Sensor, Controller, Actuator, dan MonitoringSystem beserta *method-method* yang diperlukan.

- Prinsip deteksi kebocoran arus dengan formula $I_{\text{bocor}} = I_{\text{fasa}} - I_{\text{netral}}$ berhasil diimplementasikan dan terbukti bekerja sesuai ekspektasi pada seluruh skenario pengujian, dengan *threshold* yang mengacu pada PUIL 2011.
- Empat metode komputasi numerik berhasil diterapkan: kalkulasi arus bocor (diferensial), Moving Average MA-5 (*smoothing* sensor), standar deviasi sampel (monitoring kestabilan), dan akumulasi energi (integrasi Euler).
- *Dashboard* web *real-time* berhasil diintegrasikan menggunakan `tiny_http` dan *concurrency Arc<Mutex<MonitoringSystem>>*, memungkinkan pemantauan dari *browser* tanpa menghentikan input terminal.
- Seluruh *bug* kritis yang ditemukan selama pengembangan – termasuk masalah isolasi *state* antar cabang saat injeksi, kegagalan validasi pasca-reset simulasi, dan kondisi *baseline* yang salah – berhasil diidentifikasi dan diperbaiki secara sistematis.
- Sistem ini siap dikembangkan lebih lanjut ke arah integrasi *hardware* ESP32 + SCT-013 untuk *deployment* nyata di rumah tangga atau UMKM.